

Sistema di Gestione degli Interventi di Manutenzione in un Condominio

Progetto di Programmazione e Strutture Dati — A.A. 2025/2026
Traccia 1 — Gruppo da 3 persone

Autori del progetto

Autore	Matricola	Modulo
Assuntapia Piccolo	NF12100574	Richieste & Priority Queue
Fabiana Sasso	NF12100245	Tecnici & Lista Concatenata
Lucia Schettino	NF12100025	Schedule & BST

Descrizione

Il progetto si occupa di gestire degli interventi di manutenzione all'interno di un condominio mediante la gestione delle richieste e dei tecnici addetti. A seconda della disponibilità o meno di un tecnico adatto ad una determinata richiesta viene pianificato un intervento. Nel corso del tempo è possibile aggiornare lo stato di una richiesta e di un intervento. Inoltre è possibile monitorare il carico di lavoro di ciascun tecnico, ricercare e visualizzare le richieste e gli interventi mediante dei filtri, e visualizzare lo storico degli interventi.

Informazioni di sistema

Sistema operativo	Linux (Ubuntu 22.04 LTS)
Standard C	C99 (-std=c99)
Compilatore	GCC
Build system	Makefile
Repository	[link GitHub pubblico]
Deadline consegna	29 maggio 2026

Il progetto è compilato e testato su Linux. Il Makefile è configurato per compilare separatamente il programma principale (main) e il programma di test (test). Non sono richieste librerie esterne oltre alla libreria standard C (libc).

1. Come usare e interagire con il sistema

Il programma si avvia da terminale tramite il Makefile fornito. Sono disponibili due eseguibili: il programma principale (main) e il programma di test automatizzato (test). Per avviare il programma eseguire **./condominio**

1.1 Compilazione

Nella directory radice del progetto eseguire:

```
make      # compila il programma principale
make test  # compila il programma di test
make clean # rimuove i file oggetto e gli eseguibili
```

1.2 Menu principale

Il programma principale presenta un menu interattivo da terminale organizzato nelle seguenti sezioni:

- **Gestione Richieste:** inserimento, visualizzazione per stato/urgenza/area/tipologia/tecnico, ricerca per codice, aggiornamento stato.
- **Gestione Tecnici:** registrazione nuovo tecnico, visualizzazione lista, monitoraggio carico di lavoro.
- **Pianificazione Interventi:** assegnazione richiesta a tecnico, pianificazione con verifica conflitti di orario, aggiornamento stato nel BST.
- **Visualizzazione Urgenze:** stampa delle richieste attive ordinate per livello di urgenza decrescente tramite la Priority Queue.
- **Report:** storico interventi conclusi, statistiche tipologia, tempo medio di completamento, tecnico più attivo, area più problematica.

1.3 Esecuzione test

Il programma di test supporta due modalità di avvio:

```
make test
```

```
./tests/esegui_test TC1 tests/TC1_input.txt tests/TC1_oracle.txt tests/TC1_output.txt
```

Il confronto avviene riga per riga tra l'output prodotto e il file oracolo atteso. I risultati (PASS/FAIL) vengono scritti nel file **result.txt**.

2. Motivazione della scelta degli ADT

Il sistema gestisce tre entità principali — richieste, tecnici e interventi pianificati — ognuna con esigenze operative distinte. La scelta degli ADT è stata guidata dall'analisi delle operazioni dominanti richieste dal dominio del problema.

2.1 Modulo Tecnici — Lista Concatenata Semplice

Il modulo tecnici (*tecnici.h* / *tecnici.c*) utilizza una lista concatenata semplice (*ListaTecnici*), in cui ogni nodo incapsula un puntatore a Tecnico, il numero di richieste assegnate e la lista delle stesse.

Operazioni dominanti: inserimento di un nuovo tecnico, ricerca per nome/specializzazione, scorrimento per trovare il tecnico con minor carico.

Analisi della complessità:

Operazione	Lista Concatenata (scelta)	Array Dinamico
Inserimento in testa	$O(1)$	$O(n)$ in caso di resize
Ricerca per specializzazione	$O(n)$	$O(n)$
Ricerca tecnico min-carico	$O(n)$	$O(n)$
Overhead memoria	Solo puntatore next	Capacità $\geq$ elementi

La lista concatenata è preferita all'array dinamico perché il numero di tecnici è variabile e soggetto ad aggiornamenti frequenti. L'inserimento in testa avviene in $O(1)$ senza alcun costo di ridimensionamento (realloc + copia), che in un array dinamico richiede $O(n)$ quando la capacità massima è raggiunta. Poiché la ricerca è comunque lineare in entrambe le strutture, la lista risulta la scelta ingegneristica più adeguata.

2.2 Modulo Richieste — Lista Collegata + Coda a Priorità (Max-Heap)

Il modulo richieste (*richieste.h* / *richieste.c*) combina due strutture dati complementari:

- **Lista collegata (Richiesta*):** memorizza tutte le richieste in ordine cronologico di inserimento (inserimento in coda). Supporta filtri multipli (per stato, urgenza, area, tecnico, tipologia) e ricerca ricorsiva per codice.

- **Coda a Priorità basata su Max-Heap (PriorityQueue):** gestisce l'estrazione efficiente della richiesta con urgenza massima. L'heap è implementato come array 1-based di puntatori a Richiesta (MAX_HEAP = 200 elementi).

Analisi della complessità per la Priority Queue:

Operazione	Max-Heap (scelta)	Lista Ordinata
insertPQ — inserimento	$O(\log n)$ — upheap	$O(n)$ — scorrimento
deleteMax — estrazione urgente	$O(\log n)$ — downheap	$O(1)$ — rimozione in testa
peekMax — consultazione	$O(1)$	$O(1)$
Spazio	$O(n)$ — array contiguo	$O(n)$ — con puntatori next

Il requisito esplicito della traccia di gestire le richieste per "livello di urgenza" rende la Coda a Priorità la scelta ingegneristica ottimale. Mentre una lista ordinata richiederebbe $O(n)$ per ogni inserimento mantenendo l'ordine, l'heap garantisce inserimento ed estrazione dell'elemento più urgente in $O(\log n)$. La lista collegata parallela mantiene la vista cronologica completa, necessaria per i filtri e il monitoraggio dello storico.

2.3 Modulo Schedule — Albero Binario di Ricerca (BST)

Il modulo di pianificazione (*schedule.h / schedule.c*) utilizza un BST (*Binary Search Tree*) come tipo di dato opaco (*Schedule*), in cui ogni nodo memorizza una struttura Data (anno, mese, giorno, ora_inizio, ora_fine), una copia della Richiesta associata e un puntatore al Tecnico assegnato.

La chiave di ordinamento è la data/fascia oraria dell'intervento, confrontata tramite la funzione privata *confrontaDate*, che restituisce -1, 0 o 1 a seconda che le due fasce siano non sovrapposte (precedente), sovrapposte (conflitto) o successive.

Analisi della complessità:

Operazione	BST (scelta)	Lista Ordinata
planIntervento — inserimento	$O(\log n)$ caso medio	$O(n)$ — scorrimento
verificaConflitti — ricerca conflitto	$O(\log n)$ caso medio	$O(n)$

Operazione	BST (scelta)	Lista Ordinata
stampaStorico — visita in-order	$O(n)$	$O(n)$
cercaIntervento — per codice	$O(n)$ — visita completa	$O(n)$
Caso peggiore (albero degenerare)	$O(n)$	—

Il BST è la struttura d'elezione per insiemi dinamicamente ordinati. La verifica dei conflitti di pianificazione si traduce nella ricerca di una sovrapposizione di chiave (data/fascia oraria): il BST scende a sinistra se il nuovo intervento è precedente all'attuale, a destra se è successivo e segnala conflitto se le fasce si sovrappongono (confrontaDate restituisce 0). Questo riduce la complessità da $O(n)$ di una lista a $O(\log n)$ nel caso medio. La visita simmetrica (in-order) restituisce intrinsecamente gli interventi in ordine cronologico, soddisfacendo il requisito di generazione dello storico ordinato senza costo aggiuntivo.

3. Progettazione

3.1 Architettura del sistema

Il sistema è organizzato in moduli indipendenti, ognuno responsabile di una singola entità del dominio. La dipendenza tra moduli è unidirezionale e non ciclica:

- **tipi.h** — definizioni comuni (enum Specializzazione, StatoRichiesta, TipoFiltro). Nessuna dipendenza.
- **utile.h / utile.c** — funzioni di I/O robuste (leggiIntero, leggiRiga, leggiData, flushInput). Dipende solo da tipi.h.
- **richieste.h / richieste.c** — ADT Lista Collegata + ADT Priority Queue. Dipende da tipi.h e utile.h.
- **tecnici.h / tecnici.c** — ADT Lista Concatenata Tecnici. Dipende da richieste.h, tipi.h, utile.h.
- **schedule.h / schedule.c** — ADT BST Pianificazione. Dipende da richieste.h e tecnici.h.

Il tipo Schedule è implementato come tipo opaco: la struttura `node_schedule` è definita solo in `schedule.c` e non è visibile agli altri moduli, garantendo l'incapsulamento dell'implementazione.

3.2 Strutture dati principali

Richiesta (nodo della lista e dell'heap)

Campo	Tipo	Descrizione
codice	int	Identificativo univoco (generato da <code>prossimoCodice</code>)
area	char[100]	Appartamento o zona interessata
tipologia	Specializzazione	Enum: IDRAULICO ELETTRICISTA MURATORE ASCENSORISTA GENERICO
descrizione	char[100]	Testo descrittivo del problema
data	char[11]	Data di apertura (GG/MM/AAAA)
data_chiusura	char[11]	Compilata solo al raggiungimento di CONCLUSA
tecnico	char[100]	Nome del tecnico assegnato (vuoto se non ancora assegnato)

Campo	Tipo	Descrizione
urgenza	int	Livello 1–5 (chiave della Priority Queue)
stato	StatoRichiesta	Enum: APERTA PIANIFICATA IN_LAVORAZIONE CONCLUSA ANNULLATA
next	Richiesta*	Puntatore al nodo successivo nella lista collegata

Tecnico

Campo	Tipo	Descrizione
codice_ID	char[10]	Codice numerico di 9 cifre
nome	char*	Allocato dinamicamente
specializzazione	Specializzazione	Enum — determina compatibilità con la richiesta
disponibile	bool	Flag di disponibilità

Data (chiave del BST)

Campo	Tipo	Descrizione
anno	int	Anno dell'intervento
mese	int	Mese dell'intervento
giorno	int	Giorno dell'intervento
ora_inizio	int	Ora di inizio della fascia
ora_fine	int	Ora di fine della fascia

3.3 Ciclo di vita di una richiesta

Il ciclo di vita di una richiesta segue la macchina a stati seguente. Le transizioni sono applicate dalla funzione *transizioneValida* e gestite da *aggiornaStato*. Gli stati CONCLUSA e ANNULLATA sono finali e non reversibili.

APERTA → PIANIFICATA → IN_LAVORAZIONE → CONCLUSA

Da qualsiasi stato intermedio è possibile passare ad ANNULLATA

3.4 Interazione tra i moduli

Quando l'utente inserisce una nuova richiesta, questa viene creata con *creaRichiesta* (modulo richieste), inserita nella lista collegata tramite *inserisciRichiesta* e inserita nella Priority Queue tramite *insertPQ* per la gestione delle urgenze.

L'assegnazione avviene tramite *assegnaRichiesta* (modulo tecnici), che scorre la lista dei tecnici alla ricerca del primo compatibile per specializzazione con minor carico. La funzione aggiorna il campo tecnico della Richiesta e incrementa il contatore *n_richieste* nel nodo della lista tecnici.

La pianificazione avviene tramite *planIntervento* (modulo schedule): prima si invoca *verificaConflitti* per controllare che la fascia oraria sia libera nel BST; se non ci sono conflitti, il nodo viene inserito nell'albero e lo stato della richiesta viene portato a PIANIFICATA sia nel BST (copia della struct) che nella lista collegata tramite *aggiornaStatoDaCodice*.

4. Specifica sintattica e semantica

Di seguito si riportano le specifiche delle operazioni principali sugli ADT implementati, con indicazione di firma, pre-condizioni, post-condizioni e valore di ritorno.

4.1 ADT Lista Collegata — Modulo Richieste

creaRichiesta

Firma	Richiesta* creaRichiesta(int codice, const char* area, Specializzazione tipologia, const char* descrizione, const char* data, int urgenza)
Pre-condizioni	area, descrizione, data $\neq$ NULL; urgenza $\in$ [1,5]
Post-condizioni	Restituisce un puntatore a Richiesta con stato=APERTA, next=NULL, data_chiusura="", tecnico="". Restituisce NULL in caso di fallimento malloc.
Effetti collaterali	Alloca memoria heap (sizeof(Richiesta))

inserisciRichiesta

Firma	void inserisciRichiesta(Richiesta** testa, Richiesta* nuova)
Pre-condizioni	testa $\neq$ NULL; nuova $\neq$ NULL
Post-condizioni	nuova è l'ultimo nodo della lista; l'ordine cronologico di inserimento è preservato
Effetti collaterali	Nessuno (modifica solo i puntatori della lista)

aggiornaStato / aggiornaStatoDaCodice

Firma	int aggiornaStato(Richiesta* r, StatoRichiesta nuovoStato) int aggiornaStatoDaCodice(Richiesta* testa, int codice, StatoRichiesta nuovoStato)
Pre-condizioni	r $\neq$ NULL (o lista $\neq$ NULL per la versione per codice)
Post-condizioni	Restituisce 1 e aggiorna r->stato se la transizione è valida; restituisce 0 altrimenti. La funzione transizioneValida garantisce che CONCLUSA e ANNULLATA siano stati finali.
Effetti collaterali	Nessuno oltre alla modifica di r->stato

cercaPerCodice

Firma	Richiesta* cercaPerCodice(Richiesta* testa, int codice)
Pre-condizioni	Nessuna (lista può essere NULL)
Post-condizioni	Restituisce il puntatore al nodo con codice uguale a codice, oppure NULL se non presente. Implementazione ricorsiva: caso base lista vuota o codice trovato; caso ricorsivo visita il nodo successivo.
Effetti collaterali	Nessuno — nessuna modifica alla lista

liberaListaRichieste

Firma	void liberaListaRichieste(Richiesta* testa)
Pre-condizioni	Nessuna (lista può essere NULL)
Post-condizioni	Tutti i nodi della lista sono deallocati (approccio ricorsivo post-order: prima il resto della lista, poi il nodo corrente)
Effetti collaterali	Dealloca tutta la memoria dei nodi della lista

4.2 ADT Priority Queue (Max-Heap) — Modulo Richieste

initPQ / emptyPQ / fullPQ

Firma	void initPQ(PriorityQueue* pq) int emptyPQ(const PriorityQueue* pq) int fullPQ(const PriorityQueue* pq)
Pre-condizioni	pq ≠ NULL
Post-condizioni	initPQ: pq->size = 0. emptyPQ: restituisce 1 se size=0. fullPQ: restituisce 1 se size≥MAX_HEAP (200).
Effetti collaterali	initPQ modifica pq; emptyPQ e fullPQ non modificano la struttura

insertPQ

Firma	int insertPQ(PriorityQueue* pq, Richiesta* r)

Pre-condizioni	pq ≠ NULL; r ≠ NULL; !fullPQ(pq)
Post-condizioni	r è inserita come foglia in posizione heap[++size]; l'invariante Max-Heap è ripristinato tramite upheap (sali). Restituisce 1 se successo, 0 se coda piena.
Effetti collaterali	pq->size incrementa di 1; la posizione dei puntatori nell'array heap può cambiare

deleteMax

Firma	Richiesta* deleteMax(PriorityQueue* pq)
Pre-condizioni	pq ≠ NULL; !emptyPQ(pq)
Post-condizioni	La radice heap[1] (urgenza massima) è rimossa e restituita. L'ultima foglia è spostata in radice e l'invariante Max-Heap è ripristinato tramite downheap (scendi). pq->size decrementa di 1. Restituisce NULL se coda vuota.
Effetti collaterali	pq->size decrementa; i puntatori nell'array heap sono riorganizzati

4.3 ADT Lista Concatenata — Modulo Tecnici

aggiungiTecnico / aggiungiTecnicoDiretto

Firma	ListaTecnici aggiungiTecnico(ListaTecnici testa, FILE* f) ListaTecnici aggiungiTecnicoDiretto(ListaTecnici testa, Tecnico* t)
Pre-condizioni	aggiungiTecnico: f ≠ NULL (stream di input). aggiungiTecnicoDiretto: t ≠ NULL con campi valorizzati.
Post-condizioni	Un nuovo nodo è inserito in testa alla lista. Il nodo contiene il Tecnico, n_richieste=0 e richieste_assegnate=NULL. Restituisce la nuova testa.
Effetti collaterali	Alloca memoria per il nodo e per t->nome (aggiungiTecnico)

assegnaRichiesta

Firma	ListaTecnici assegnaRichiesta(ListaTecnici testa, Specializzazione specializzazione, Richiesta* r)
Pre-condizioni	$r \neq \text{NULL}$
Post-condizioni	Trova il tecnico compatibile (stessa specializzazione o GENERICO) con minor n_richieste tramite trovaTecnico; aggiorna r->tecnico con il nome del tecnico; incrementa n_richieste del nodo; inserisce la richiesta nella lista richieste_assegnate del nodo. Restituisce la lista aggiornata, o NULL se nessun tecnico disponibile.
Effetti collaterali	Modifica r->tecnico e il nodo del tecnico scelto

liberaLista

Firma	void liberaLista(ListaTecnici testa)
Pre-condizioni	Nessuna (può essere NULL)
Post-condizioni	Tutti i nodi della lista, inclusi i dati dei Tecnico e le stringhe nome allocate dinamicamente, sono deallocati.
Effetti collaterali	Dealloca tutta la memoria della struttura

4.4 ADT BST – Modulo Schedule

verificaConflitti

Firma	int verificaConflitti(Schedule root, Data target)
Pre-condizioni	target.ora_fine > target.ora_inizio
Post-condizioni	L'albero non è modificato. Restituisce 1 se esiste un nodo nell'albero la cui fascia si sovrappone a target (confrontaDate = 0), 0 altrimenti. La ricerca segue l'invariante BST: scende a sinistra se target è precedente, a destra se è successivo.
Effetti collaterali	Nessuno

planIntervento

Firma	Schedule planIntervento(Schedule root, Richiesta req, Tecnico* tec, Data data)
Pre-condizioni	data.ora_fine > data.ora_inizio; verificaConflitti(root, data) = 0
Post-condizioni	Un nuovo nodo è inserito nell'albero nella posizione corretta rispetto all'ordinamento per data. req.stato è impostato a PIANIFICATA nel nodo inserito. Restituisce la nuova radice.
Effetti collaterali	Alloca memoria per il nuovo nodo (sizeof(struct node_schedule))

stampaStorico

Firma	void stampaStorico(Schedule root, FILE* out)
Pre-condizioni	out ≠ NULL; i nodi con stato CONCLUSA hanno tec ≠ NULL
Post-condizioni	Stampa su out gli interventi con stato CONCLUSA in ordine cronologico (visita in-order). L'albero non è modificato.
Effetti collaterali	Output su file/stdout

liberaAlbero

Firma	void liberaAlbero(Schedule root)
Pre-condizioni	Nessuna (può essere NULL)
Post-condizioni	Tutti i nodi dell'albero sono deallocati (visita post-order). I puntatori Tecnico* contenuti nei nodi non sono deallocati (gestiti dal modulo tecnici).
Effetti collaterali	Dealloca tutta la memoria dei nodi BST

5. Razionale dei casi di test

Il framework di test è automatizzato: per ogni caso di test (TC) viene letto un file di input (TCn_input.txt), prodotto un output (TCn_output.txt) e confrontato riga per riga con un oracolo (TCn_oracle.txt). L'esito (PASS/FAIL) è scritto in result.txt. La test suite completa comprende 13 casi di test organizzati per modulo.

5.1 Modulo Richieste

ID	Scenario	Obiettivo	Criterio di accettazione
TC1	Registrazione richiesta valida	Verifica che creaRichiesta allochi correttamente e inserisciRichiesta aggiunga in coda	Output: codice=1, area corretta, stato=APERTA
TC8	Aggiornamento stato — transizione valida	Verifica che aggiornaStatoDaCodice aggiorni lo stato per una transizione lecita (APERTA→PIANIFICATA)	Output: STATO AGGIORNATO: PIANIFICATA
TC9	Aggiornamento stato — transizione invalida	Verifica che una transizione illegale (CONCLUSA→qualsiasi) sia rifiutata	Output: TRANSIZIONE NON PERMESSA
TC10	Ricerca per codice — trovata	Verifica che cercaPerCodice restituisca il nodo corretto	Output: TROVATA codice=N
TC11	Ricerca per codice — non trovata	Verifica che cercaPerCodice restituisca NULL per un codice inesistente	Output: NON TROVATA

5.2 Modulo Tecnici

ID	Scenario	Obiettivo	Criterio di accettazione
TC2	Registrazione tecnico	Verifica che aggiungiTecnico crei correttamente il nodo con i dati da input	Output: codice, nome, specializzazione, richieste: 0
TC3	Assegnazione richiesta — match diretto	Verifica che assegnaRichiesta assegni al tecnico con la stessa specializzazione e minor carico	Output: ASSEGNAZIONE OK tecnico=<nome>

ID	Scenario	Obiettivo	Criterio di accettazione
TC4	Assegnazione con fallback GENERICO	Verifica che un tecnico GENERICO possa ricevere qualsiasi tipologia di richiesta	Output: ASSEGNAZIONE OK tecnico=<nome>
TC5	Assegnazione fallita — lista vuota	Verifica che se non vi sono tecnici disponibili il campo tecnico rimanga vuoto	Output: NESSUN TECNICO DISPONIBILE

5.3 Modulo Schedule

ID	Scenario	Obiettivo	Criterio di accettazione
TC6	Pianificazione senza conflitti	Verifica che planIntervento inserisca nell'albero una fascia oraria libera	Output: NESSUN CONFLITTO → PIANIFICAZIONE OK
TC7	Pianificazione con conflitto	Verifica che verificaConflitti rilevi la sovrapposizione tra due interventi con stessa fascia	Output: PRIMO INTERVENTO PIANIFICATO → CONFLITTO RILEVATO
TC12	Storico interventi conclusi	Verifica che stampaStorico emetta solo gli interventi con stato CONCLUSA in ordine cronologico	Output: riga formattata con data, tecnico, codice
TC13	Generazione report completo	Verifica generaReport (aperti/conclusi) e tempoMedioCompletamento su un albero con dati noti	Output: Aperti: N, Conclusi: M, Tempo medio: K giorni

5.4 Note sulla strategia di test

I test sono progettati per coprire sia i percorsi attesi sia i casi limite: lista vuota, codice inesistente, transizione di stato invalida, conflitto di pianificazione, lista tecnici vuota. L'approccio black-box confronta l'output del programma con un oracolo predefinito, garantendo la riproducibilità automatica su qualsiasi piattaforma.

Il confronto avviene tramite la funzione *confrontaFile* in *test_main.c*, che legge i due file riga per riga e segnala FAIL alla prima discrepanza. Questo assicura che il test verifichi sia la correttezza del risultato che il formato esatto dell'output.